Benazir Bhutto Shaheed Human Resource Research & Development Board [BBSHRRDB]
PMU-ORIC University of Sindh, Jamshoro

Django Lab Manual: Building a Post Model, Django Admin & Data Seeding with Faker

Lab Overview

This lab teaches you how Django's three core layers — **Models**, **Migrations**, and **Admin** — work together, and how to populate a database quickly using **Faker**, a fake-data generation library. By the end, you will understand not just *how* to run each command, but *why* it exists and *what happens internally* when you run it.

Learning Objectives

By the end of this lab, you will be able to:

- Explain what a Django Model is and how it maps to a database table
- Understand each Django field type and its underlying database behavior
- Explain the two-step migration system and why it's split into two commands
- Customize the Django Admin interface using `ModelAdmin`
- Use Faker to generate realistic dummy data programmatically
- Understand Django's standalone script setup process (`django.setup()`)

Prerequisites

- A working Django project with at least one app registered in `INSTALLED_APPS`
- Python 3.8+ installed

- Basic familiarity with the terminal/command prompt
- `pip` package manager available

Part 1: Understanding Django Models

1.1 What Is a Model?

In Django, a **Model** is a Python class that represents a database table. This is called an **ORM** — Object-Relational Mapper. Instead of writing raw SQL like:

```sql
CREATE TABLE post (
    id INTEGER PRIMARY KEY,
    title VARCHAR(200),
    content TEXT,
    ...
);
```

...you write a Python class, and Django translates it into SQL for you. Every model class you write **must inherit** from `models.Model`. This inheritance is what tells Django "this class represents a database table," giving it access to Django's ORM machinery — saving, querying, filtering, deleting, etc.

1.2 Breaking Down the Post Model

```python
from django.db import models


class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    author = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)
    is_published = models.BooleanField(default=False)


    def __str__(self):
        return self.title
```

Field-by-field explanation:

Field	Type	Why it's used	What it does under the hood
`title`	`CharField(max_length=200)`	Short text with a defined limit	Maps to `VARCHAR(200)` in SQL. `max_length` is **required** for `CharField` and enforced at both the

Field	Type	Why it's used	What it does under the hood
			database and form-validation level
`content`	`TextField()`	Long-form text with no fixed limit	Maps to `TEXT` in SQL. No `max_length` needed since it's meant for large content like paragraphs
`author`	`CharField(max_length=100)`	Short text (a name)	Same as `title`, just a smaller limit since names are short
`created_at`	`DateTimeField(auto_now_add=True)`	Automatic timestamp	Django sets this field's value **only once**, at the moment the object is first created, and never again — even if you update the record later. This is different from `auto_now=True`, which updates the timestamp **every time** you save
`is_published`	`BooleanField(default=False)`	True/False flag	Maps to a boolean column. `default=False` means any new post

Field	Type	Why it's used	What it does under the hood
			starts as a "draft" unless explicitly published

Why `__str__` matters: Without `__str__`, if you open the Django Admin or a Python shell and print a `Post` object, you'd see something unhelpful like `Post object (1)`. Defining `__str__` tells Python (and Django) how to represent the object as a human-readable string — here, showing the post's title instead of a generic label.

Part 2: Migrations — How Django Talks to the Database

Migrations are Django's **version control system for your database schema**. Every time you add, remove, or change a field in a model, the actual database table needs to be updated to match. Migrations automate this instead of forcing you to hand-write `ALTER TABLE` SQL statements.

2.1 Step 1 — python manage.py makemigrations

This command does **not** touch your database. Instead, it:

1. Scans all your installed apps' `models.py` files
2. Compares your current models against the **last known migration state** (Django tracks this internally)
3. Detects differences (a new model, a new field, a changed field type, etc.)
4. Generates a **migration file** — a Python file inside your app's `migrations/` folder that contains instructions (like a blueprint) describing exactly what changed

Think of `makemigrations` as writing a **construction plan**, not building anything yet.

2.2 Step 2 — python manage.py migrate

This command reads the migration files (the "plans") generated above and **actually executes them** against your database — creating tables, adding columns, or modifying schema as needed.

Why two separate steps instead of one? This separation gives you a chance to **review the migration file** before it touches your real database. It also allows migrations to be version-controlled (committed to Git) and shared across a team, so everyone's database schema stays in sync without directly sharing database files.

2.3 Creating a Superuser

To log into the admin panel, you need an account with elevated permissions:

```shell
python manage.py createsuperuser
```

This command interactively prompts for a username, email, and password, then creates an admin-level user record directly in the `auth_user` table (or your custom user model, if configured).

Part 3: Django Admin — Registering and Customizing

3.1 Why Registration Is Needed

Django Admin doesn't automatically show every model you create — this is intentional, giving you control over what's exposed. You must explicitly **register** a model in `admin.py` for it to appear in the admin dashboard.

3.2 Breaking Down the Admin Configuration

```python
from django.contrib import admin

from .models import Post


@admin.register(Post)

class PostAdmin(admin.ModelAdmin):

    list_display = ('title', 'author', 'is_published', 'created_at')

    list_filter = ('is_published', 'created_at')

    search_fields = ('title', 'author')
```

Concept breakdown:

- **`@admin.register(Post)`** — This is a *decorator*, a shorthand equivalent to writing `admin.site.register(Post, PostAdmin)` separately. It links your customization class (`PostAdmin`) to the `Post` model.

- **`ModelAdmin` subclass** — By default, the admin list view for any model just shows a single column with the `__str__` output. Subclassing `ModelAdmin` lets you override this default behavior.

- **`list_display`** — Controls which fields appear as **columns** in the admin's list view (the table you see when you click into "Posts"). Without this, you'd only see one column.

- `list_filter` — Adds a **sidebar filter panel** on the right of the list view. Django automatically generates filter options based on the field's values (e.g., True/False for booleans, or date ranges for `DateTimeField`).

- `search_fields` — Adds a **search bar** at the top of the list view. Django performs a `LIKE` query (partial text match) across the specified fields whenever you type a search term.

Part 4: Seeding Data with Faker

4.1 What Is Faker?

Faker is a third-party Python library that generates realistic **fake data** — names, sentences, paragraphs, addresses, dates, etc. It's commonly used for testing, populating demo databases, and load-testing without needing real user data.

Install it with:

```shell
pip install faker
```

4.2 Why a Standalone Script Needs django.setup()

Normally, Django code runs *inside* the framework's request-response cycle, where settings and apps are already loaded automatically. But when you write a standalone script (like `seed_posts.py`) that isn't run through `manage.py`, Django has no idea which project or settings to use.

```python
import os

import django


os.environ.setdefault('DJANGO_SETTINGS_MODULE',
'myproject.settings')

django.setup()
```

What each line does:

- `os.environ.setdefault(...)` — Tells Python's environment which settings module to use, mimicking what `manage.py` normally does automatically behind the scenes
- `django.setup()` — Manually triggers Django's internal app-loading process (registering models, configuring the ORM, etc.), which is normally handled automatically when you run the dev server

Critical rule: These two lines must run **before** you import any models — otherwise Django won't be ready and you'll get an `AppRegistryNotReady` error.

4.3 Understanding the Seeding Logic

```python
from myapp.models import Post

from faker import Faker


fake = Faker()
```

```
def generate_fake_posts(number_of_posts=200):
    for i in range(number_of_posts):
        fake_title = fake.sentence(nb_words=6)
        fake_content = fake.paragraph(nb_sentences=5)
        fake_author = fake.name()
        fake_is_published = random.choice([True, False])

        Post.objects.create(
            title=fake_title,
            content=fake_content,
            author=fake_author,
            is_published=fake_is_published
        )
```

Key concepts:

- **`Faker()` instance** — Creates a generator object. Every call like `fake.name()` or `fake.sentence()` produces a new random value each time.
- **`fake.sentence(nb_words=6)`** — Generates a random sentence with approximately 6 words, used here to fake a post title.
- **`fake.paragraph(nb_sentences=5)`** — Generates a paragraph made of 5 random sentences, used as fake post content.

- `fake.name()` — Generates a realistic-looking full name.
- `random.choice([True, False])` — Randomly selects one of the two boolean values, simulating that some posts are published and others are drafts.
- `Post.objects.create(...)` — This is Django's ORM shortcut that both **instantiates** a new `Post` object *and* **saves it to the database** in a single call (equivalent to calling `Post(...)` followed by `.save()`).

4.4 Running the Script

Unlike Django management commands, this script is executed directly with plain Python:

```
Shell
python seed_posts.py
```

This works because of the `if __name__ == '__main__':` guard at the bottom of the script, which ensures `generate_fake_posts(200)` only runs when the file is executed directly (not when imported elsewhere).

Part 5: Lab Tasks

Complete the following tasks in order. Do not skip steps — each part depends on the previous one.

Task 1 — Model Creation

Create a `Post` model inside your app's `models.py` with the following exact fields:

- `title` (short text)
- `content` (long text)
- `author` (short text)

- `created_at` (auto-filled date and time)
- `is_published` (boolean, default False)

Add a `__str__` method that returns the post's title.

Task 2 — Migrations

- Run `python manage.py makemigrations` and confirm a new migration file is created inside your app's `migrations/` folder.
- Run `python manage.py migrate` and confirm the `Post` table is created without errors.

Task 3 — Superuser Setup

- Run `python manage.py createsuperuser` and create an admin account.

Task 4 — Admin Registration

- Register the `Post` model in `admin.py` using `@admin.register(Post)`.
- Configure `list_display` to show `title`, `author`, `is_published`, and `created_at`.
- Add `list_filter` for `is_published` and `created_at`.
- Add `search_fields` for `title` and `author`.

Task 5 — Install Faker

- Install the Faker package using `pip install faker`.

Task 6 — Write the Seeding Script

- Create `seed_posts.py` in your project root (same folder as `manage.py`).
- Write the script to generate 200 fake `Post` records, replacing `myproject.settings` and `myapp.models` with your actual project/app names.

Task 7 — Run and Verify

- Run the seeding script: `python seed_posts.py`.
- Start your server: `python manage.py runserver`.
- Log into `/admin` with your superuser account.

Submission Checklist

- [ ] `makemigrations` and `migrate` ran without errors
- [ ] Superuser account created and can log into `/admin`
- [ ] "Posts" section visible in the admin panel
- [ ] Admin list view shows columns for Title, Author, Published, and Date Created
- [ ] Search bar and sidebar filters work correctly
- [ ] Exactly 200 fake posts exist in the database